

2. Healthcare: Patient Analytics and Pandemic Response

Healthcare providers use Python's data visualization capabilities to analyze patient demographics, track treatment outcomes, and respond to public health challenges. During the COVID-19 pandemic, Python-based visualizations became critical for monitoring infection rates, resource allocation, and population density impacts on disease transmission.

What is a SubPlot,

- A **subplot** means **multiple graphs inside one single figure (canvas)**.
- Instead of showing one chart per window, we divide the screen into a grid and place different charts in each section.

Example idea to explain in class:

- Imagine a **2 × 2 grid (like a table)**
- Each cell contains a different chart

[Chart 1 | Chart 2]

[Chart 3 | Chart 4]

Why use subplots?

- Compare multiple datasets at once
- Save space
- Build dashboards (like this example)

Code for Create subplot,

```
make_subplots(rows=2, cols=2)
```

This creates a **2-row, 2-column layout (4 plots total)**

Step 1: Import libraries

```
import numpy as np

import pandas as pd

import plotly.graph_objects as go

from plotly.subplots import make_subplots
```

Step 2: Generate random data

```
ng = np.random.default_rng(42)
```

Patient Dataset

```
# --- Simulated data ---

patient_data = pd.DataFrame({

    "age": rng.integers(18, 90, 300),

    "treatment_duration": rng.gamma(2.0, 5.0, 300).astype(int).clip(1, None),

    "recovery_rate": np.clip(np.round(rng.normal(0.85, 0.08, 300), 2), 0.4, 1.0),

})
```

Geographic dataset

```
geo_data = pd.DataFrame({

    "state": ["CA", "TX", "NY", "FL", "IL", "PA", "OH", "GA", "NC", "MI"],

    "case_rate": np.round(rng.uniform(100, 2000, 10), 2)

})
```

Step 3: Create subplots layout

```
# --- Layout ---

fig = make_subplots(

    rows=2, cols=2,

    subplot_titles=('Patient Age Distribution', 'Treatment Outcomes',

                    'Geographic Spread', 'Resource Utilization'),

    specs=[[{'type': 'bar'}, {'type': 'scatter'}],

           [{'type': 'geo'}, {'type': 'pie'}]]

)
```

Position	Chart Type
(1,1)	Bar/Histogram
(1,2)	Scatter
(2,1)	Map
(2,2)	Pie

Step 4: Add charts (traces)

Histogram (Age distribution)

```
fig.add_trace(go.Histogram(x=patient_data['age'], name='Age', showlegend=False), row=1, col=1)
```

Scatter plot (Treatment vs Recovery)

```
fig.add_trace(go.Scatter(x=patient_data['treatment_duration'], y=patient_data['recovery_rate'],

                          mode='markers', name='Outcomes', showlegend=False), row=1, col=2)
```

Map (Choropleth)

```
fig.add_trace(go.Choropleth(locations=geo_data['state'], z=geo_data['case_rate'],
                             locationmode='USA-states', colorscale='Plasma',
                             colorbar=dict(title='Cases / 100k', x=0.44, xanchor='left',
                             y=0.03, yanchor='bottom', len=0.4, thickness=12)),
              row=2, col=1)
```

Pie chart (Resource usage)

```
fig.add_trace(go.Pie(labels=['ICU', 'General', 'Emergency', 'Outpatient'],
                      values=[30, 40, 15, 15], showlegend=False, textinfo='percent+label'),
              row=2, col=2)
```

Step 5: Axis labels/Map settings

```
# --- Axes / geos ---

fig.update_xaxes(title_text="Age (years)", row=1, col=1)
fig.update_yaxes(title_text="Number of Patients", row=1, col=1)
fig.update_xaxes(title_text="Treatment Duration (days)", row=1, col=2)
fig.update_yaxes(title_text="Recovery Rate", row=1, col=2)
fig.update_geos(scope='usa', row=2, col=1)
```

```
# --- Title / margins ---

fig.update_layout(

    title='Healthcare Analytics Dashboard', height=800,

    margin=dict(t=90, r=20, l=20, b=20)

)

fig.show()
```

```
fig.show()
```

Exercise: Build a Multi-Plot Business Dashboard

Objective

Create a **6-chart dashboard** using **Seaborn** and **Matplotlib subplots** to analyze customer behavior, sales trends, and engagement.

Instructions

Step 1: Import Required Libraries

Import the following:

- NumPy
- Pandas
- Seaborn
- Matplotlib
- Matplotlib dates module

Step 2: Set Visualization Style

Set the Seaborn theme to:

- Grid-based background for better readability

Step 3: Generate Simulated Data

Create the following datasets:

Customer Data

- 400 records
- Columns:
 - age_group (4 categories)
 - frequency (purchase count)
 - monetary_value (spending)

Create a new column:

- customer_segment (Low, Mid, High) based on spending

Sales Data

- Date range starting from **2025-01-01**
- 120 days
- Revenue should show a **cumulative increasing trend**

Product Data

- Categories:
 - Electronics, Clothing, Home, Toys
- Random sales values

Social Media Data

- Platforms:
 - Twitter, Instagram, Facebook
- Content types:
 - Video, Image, Text
- Engagement rate values

Step 4: Create Subplots Layout

Create a **2 × 3 subplot grid**

This means:

- 2 rows
- 3 columns
- Total = 6 charts

Step 5: Create the Following Visualizations

1. Customer Age Demographics

- Chart type: Count plot
- X-axis: Age group

2. Purchase Behavior Analysis

- Chart type: Scatter plot
- X-axis: Purchase frequency
- Y-axis: Monetary value
- Color by customer segment

3. Conversion Funnel

- Chart type: Bar chart

- Stages:
 - Visitors
 - Product Views
 - Cart Adds
 - Purchases

4. Revenue Trend

- Chart type: Line plot
- X-axis: Date
- Y-axis: Revenue
- Format dates properly on axis

5. Category Performance

- Chart type: Bar plot
- X-axis: Product category
- Y-axis: Sales

6. Social Media Engagement

- Create a **pivot table**
- Chart type: Heatmap
- Show values inside cells

Step 6: Add Labels and Titles

Each subplot must include:

- Title
- X-axis label
- Y-axis label

Step 7: Final Touches

- Adjust layout to avoid overlapping
- Display the dashboard

Expected Output

A **single dashboard** containing:

- 6 different visualizations
- Organized in a **2×3 grid**
- Clearly labeled and readable

Hints

- Use `plt.subplots()` for layout
- Use `sns.countplot()`, `sns.scatterplot()`, `sns.lineplot()`, `sns.barplot()`, `sns.heatmap()`
- Use `pivot_table()` for heatmap data
- Use `matplotlib.dates` for formatting dates